UNIVERSITAS
DIAN NUSANTARA

# Struktur Data dan Algoritma

## Array dan Algoritma Pencarian

Fakultas : FTI

Program Studi : TEKNIK INFORMATIKA

Tatap Muka

02

Kode Matakuliah : 412523301

Disusun oleh : Riad Sahara, S.SI., M.T.

# DESKRIPSI MATERI PERTEMUAN 2

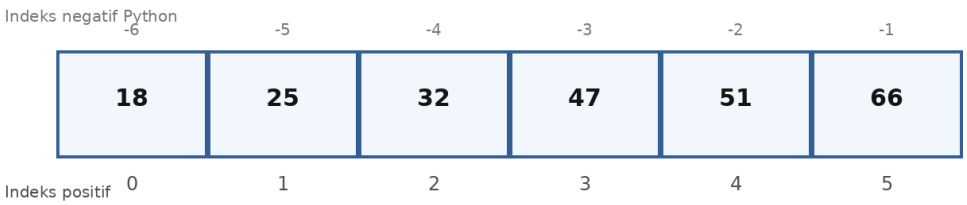
| | |
|---|---|
| Materi Pertemuan 2 | **Array dan Algoritma Pencarian** |
| Deskripsi Materi | Materi membahas konsep array, representasi indeks, operasi dasar, karakteristik Python list sebagai dynamic array, traversal, Linear Search, Binary Search, analisis kompleksitas, serta praktik implementasi langkah demi langkah menggunakan Google Colab. |
| Sub Capaian Pembelajaran Mata Kuliah (Sub CPMK) | Luaran yang diharapkan setelah mengikuti materi pada pertemuan ini:<br>• Mahasiswa mampu menjelaskan konsep array, indeks, dan karakteristik akses elemen.<br>• Mahasiswa mampu melakukan operasi akses, traversal, update, insert, dan delete pada array/list.<br>• Mahasiswa mampu mengimplementasikan Linear Search dan Binary Search menggunakan Python.<br>• Mahasiswa mampu menjelaskan syarat penggunaan Binary Search dan membandingkannya dengan Linear Search.<br>• Mahasiswa mampu menganalisis kompleksitas operasi dasar array serta algoritma pencarian.<br>• Mahasiswa mampu melakukan eksperimen sederhana untuk mengamati pengaruh ukuran input terhadap waktu pencarian. |
| Deskripsi Tugas | Latihan formatif dan praktik Google Colab meliputi manipulasi list, implementasi Linear Search, implementasi Binary Search, dry run, eksperimen waktu eksekusi, serta studi kasus pencarian data mahasiswa. |
| **Kontrak Perkuliahan** | **Ketentuan perkuliahan:**<br>1. Mahasiswa membaca modul dan membawa notebook Pertemuan 1 atau membuat notebook baru Pertemuan 2.<br>2. Seluruh contoh kode dijalankan sendiri di Google Colab, bukan hanya dibaca.<br>3. Setiap hasil praktik harus disertai penjelasan singkat mengenai proses dan kompleksitas algoritma.<br>4. Komposisi penilaian mengikuti RPS resmi dan ketentuan kelas yang berlaku. |

# Pertemuan 2
# Array dan Algoritma Pencarian

## 2.1. Pengantar Array dan Pencarian Data

Pada pertemuan sebelumnya, struktur data diperkenalkan sebagai cara mengorganisasi data agar operasi tertentu dapat dilakukan secara efektif. Pertemuan ini mulai membahas struktur data secara konkret melalui array. Array merupakan salah satu struktur data paling dasar karena menyimpan sekumpulan elemen dalam urutan tertentu dan menyediakan cara untuk mengakses elemen berdasarkan posisi atau indeks.

Konsep array penting karena banyak struktur data dan algoritma lain dibangun dari gagasan penyimpanan berurutan. Selain itu, persoalan sederhana seperti mencari NIM, nilai, kode barang, atau nomor transaksi dapat digunakan untuk melihat hubungan langsung antara struktur data, algoritma, dan kompleksitas.




Gambar 2.1 Representasi elemen dan indeks pada array/list

*Sumber: Diolah oleh penulis.*

### Tujuan Pembelajaran Pertemuan 2

- Memahami cara elemen array disimpan dan diakses melalui indeks.
- Mampu melakukan operasi dasar pada array/list menggunakan Python.
- Memahami proses traversal dan hubungannya dengan kompleksitas O(n).
- Mampu membuat Linear Search dan Binary Search dari awal.
- Mampu memilih algoritma pencarian sesuai kondisi data.
- Mampu menguji kode dan membaca hasil eksperimen sederhana menggunakan Google Colab.

Catatan penting: pada Python, struktur `list` merupakan *dynamic array*, bukan array statis seperti pada C. Dalam modul ini, `list` digunakan sebagai alat implementasi untuk mempelajari konsep array dan operasi dasarnya.

## 2.2. Konsep Dasar Array

### 2.2.1 Apa yang Dimaksud dengan Array?

Array adalah struktur data linear yang menyimpan sekumpulan elemen dalam urutan tertentu. Setiap elemen mempunyai posisi yang dapat diidentifikasi menggunakan indeks. Pada model array klasik, elemen umumnya mempunyai tipe data yang sama dan disimpan pada lokasi memori yang berdekatan. Karakteristik ini membuat akses berdasarkan indeks dapat dilakukan secara langsung.

Jika sebuah array berisi n elemen, indeks biasanya dimulai dari 0 dan berakhir pada n - 1. Dengan demikian, array berisi lima elemen memiliki indeks 0, 1, 2, 3, dan 4. Kesalahan memahami batas indeks merupakan salah satu sumber *IndexError* yang paling sering terjadi pada praktik awal.

| Posisi Logis | Indeks Python | Nilai Contoh |
|---|---|---|
| Elemen pertama | 0 | 18 |
| Elemen kedua | 1 | 25 |
| Elemen ketiga | 2 | 32 |
| Elemen terakhir | len(data)-1 | 66 |

### 2.2.2 Akses Langsung Berdasarkan Indeks

Ketika indeks diketahui, array dapat langsung menuju elemen yang diminta. Secara konseptual, alamat elemen dapat dihitung dari alamat awal array, ukuran elemen, dan indeks. Karena jumlah langkah perhitungan tidak bertambah seiring bertambahnya jumlah elemen, operasi akses berdasarkan indeks dianggap berkompleksitas O(1).

> O(1) tidak berarti waktu eksekusinya selalu tepat satu nanodetik atau satu operasi mesin. O(1) berarti jumlah pekerjaan utama tidak tumbuh mengikuti ukuran n.

### 2.2.3 Array Statis dan Dynamic Array

Array statis memiliki kapasitas yang ditetapkan ketika dibuat. Jika kapasitas penuh, ukuran tidak dapat diperbesar tanpa membuat area penyimpanan baru. Sebaliknya, *dynamic array* dapat memperbesar kapasitas internal ketika diperlukan. Python `list` menggunakan konsep *dynamic array*, sehingga `append()` dapat digunakan tanpa menentukan ukuran awal.

Kemudahan tersebut tidak berarti seluruh operasi pada `list` selalu murah. Menambahkan elemen di akhir biasanya sangat efisien, tetapi menyisipkan elemen di awal atau tengah dapat memaksa banyak elemen bergeser. Oleh karena itu, kita tetap perlu memahami proses di balik satu baris kode Python.

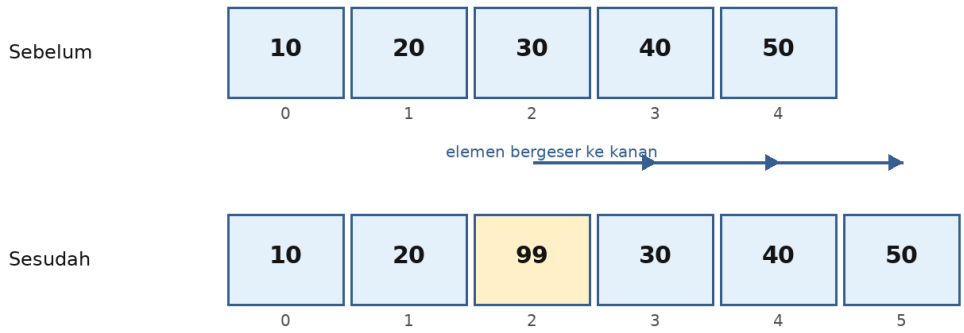
## 2.3. Operasi Dasar pada Array

Operasi dasar yang sering dilakukan pada array meliputi akses, traversal, pembaruan, penambahan, penyisipan, penghapusan, dan pencarian. Setiap operasi memiliki pola kerja dan kompleksitas yang berbeda.

| Operasi | Contoh Python | Kompleksitas Umum | Penjelasan |
|---|---|---|---|
| Akses indeks | data[i] | O(1) | Mengambil elemen pada posisi tertentu. |
| Update indeks | data[i] = x | O(1) | Mengubah elemen pada posisi tertentu. |
| Traversal | for x in data | O(n) | Mengunjungi seluruh elemen. |
| Append | data.append(x) | O(1) amortized | Menambah elemen di akhir. |
| Insert tengah | data.insert(i, x) | O(n) | Elemen setelah posisi i dapat bergeser. |
| Hapus tengah | data.pop(i) | O(n) | Elemen setelah posisi i dapat bergeser. |
| Pencarian nilai | x in data | O(n) | Pada kasus umum memeriksa elemen secara linear. |

### 2.3.1 Mengapa Insert di Tengah Memerlukan Pergeseran?

Misalkan data [10, 20, 30, 40, 50] akan disisipi nilai 99 pada indeks 2. Nilai 30, 40, dan 50 harus berpindah satu posisi ke kanan agar indeks 2 tersedia untuk 99. Semakin dekat posisi penyisipan ke awal array, semakin banyak elemen yang berpotensi bergeser.




Gambar 2.2 Ilustrasi pergeseran elemen saat insert di tengah

*Sumber: Diolah oleh penulis.*

### 2.3.2 Mengapa Append Biasanya Lebih Murah?

Pada *dynamic array*, kapasitas internal biasanya disediakan sedikit lebih besar daripada jumlah elemen yang sedang digunakan. Ketika masih tersedia ruang, `append()` cukup menempatkan elemen baru pada posisi berikutnya. Sesekali kapasitas harus diperbesar dan data dipindahkan ke area baru. Karena biaya besar tersebut tidak terjadi pada setiap `append()`, kompleksitasnya biasa disebut O(1) *amortized*.

## 2.4. Persiapan Praktik Python Pertemuan 2

Praktik tetap menggunakan Google Colab seperti pada Pertemuan 1. Tujuan bagian ini bukan mengulang seluruh persiapan lingkungan, tetapi memastikan setiap mahasiswa mempunyai notebook yang rapi dan dapat mengikuti contoh secara bertahap.

### 2.4.1 Membuat Notebook Praktik

1. Buka Google Colab melalui browser dan masuk menggunakan akun Google.
2. Pilih **New notebook** atau salin notebook Pertemuan 1 jika ingin mempertahankan catatan sebelumnya.
3. Ubah nama menjadi `SDA_Pertemuan_02_Nama_NIM.ipynb`.
4. Buat *text cell* untuk judul bagian, misalnya "A. Operasi Array", "B. Linear Search", dan "C. Binary Search".
5. Jalankan setiap contoh kode pada *code cell* terpisah agar perubahan data mudah diamati.
6. Gunakan Shift + Enter untuk menjalankan cell dan berpindah ke cell berikutnya.
7. Jika muncul error, baca jenis error dan nomor baris sebelum memperbaiki kode.

### 2.4.2 Data Awal untuk Praktik

**Contoh 2.1 Membuat list sebagai representasi array**

```
nilai = [78, 85, 90, 67, 88]
print(nilai)
print(type(nilai))
```

**Keluaran yang diharapkan:**

```
[78, 85, 90, 67, 88]
<class 'list'>
```

Baris pertama membuat sebuah objek `list` dan menyimpannya pada variabel `nilai`. Tanda kurung siku menunjukkan kumpulan elemen. Baris kedua menampilkan isi data. Baris ketiga menggunakan `type()` untuk memastikan bahwa struktur yang digunakan adalah `list`.

### 2.4.3 Mengenal `len()` dan Batas Indeks

**Contoh 2.2 Menentukan jumlah elemen dan indeks terakhir**

```
print('Jumlah elemen:', len(nilai))
print('Indeks terakhir:', len(nilai) - 1)
print('Elemen terakhir:', nilai[len(nilai) - 1])
```

**Keluaran yang diharapkan:**

```
Jumlah elemen: 5
Indeks terakhir: 4
Elemen terakhir: 88
```

Fungsi `len(nilai)` menghasilkan jumlah elemen, yaitu 5. Karena indeks dimulai dari 0, indeks terakhir adalah 4. Pola `len(data) - 1` akan sering digunakan ketika menentukan batas kanan pada Binary Search.

## 2.5. Tutorial Operasi Array dengan Python

### 2.5.1 Mengakses Elemen

**Contoh 2.3 Akses berdasarkan indeks**

```
nilai = [78, 85, 90, 67, 88]
print(nilai[0])
print(nilai[2])
print(nilai[-1])
```

**Keluaran yang diharapkan:**

```
78
90
88
```

`nilai[0]` membaca elemen pertama, `nilai[2]` membaca elemen ketiga, dan `nilai[-1]` adalah sintaks Python untuk membaca elemen terakhir. Indeks negatif berguna dalam Python, tetapi pada analisis struktur data kita tetap menggunakan indeks nonnegatif 0 sampai n - 1 agar konsep array lebih mudah dibandingkan dengan bahasa lain.

### 2.5.2 Memperbarui Nilai

**Contoh 2.4 Update elemen pada indeks tertentu**

```
nilai[3] = 75
print(nilai)
```

**Keluaran yang diharapkan:**

```
[78, 85, 90, 75, 88]
```

Operator penugasan mengubah nilai pada indeks 3 dari 67 menjadi 75. Tidak ada elemen lain yang bergeser. Karena posisi sudah diketahui, operasi ini bersifat O(1).

### 2.5.3 Menambahkan Elemen di Akhir

**Contoh 2.5 Menambahkan elemen dengan append()**

```
nilai.append(92)
print(nilai)
```

**Keluaran yang diharapkan:**

```
[78, 85, 90, 75, 88, 92]
```

Method `append()` menambahkan satu elemen pada akhir list. Pada Python, operasi ini memiliki kompleksitas O(1) secara *amortized*. Dalam praktik, `append()` sangat umum digunakan ketika data dibangun secara bertahap.

### 2.5.4 Menyisipkan Elemen pada Posisi Tertentu

**Contoh 2.6 Menyisipkan elemen**

```
nilai.insert(2, 80)
print(nilai)
```

**Keluaran yang diharapkan:**

```
[78, 85, 80, 90, 75, 88, 92]
```

`insert(2, 80)` menempatkan nilai 80 pada indeks 2. Elemen yang sebelumnya berada pada indeks 2 dan sesudahnya bergeser ke kanan. Karena jumlah elemen yang bergeser dapat bertambah seiring ukuran data, kompleksitas kasus terburuk adalah O(n).

## 2.6. Penghapusan Elemen dan Perbedaan Method

### 2.6.1 `pop()` Menghapus Berdasarkan Indeks

**Contoh 2.7 Menghapus berdasarkan indeks**

```
data = [10, 20, 30, 40, 50]
dihapus = data.pop(2)
print('Dihapus:', dihapus)
print('Data:', data)
```

**Keluaran yang diharapkan:**

```
Dihapus: 30
Data: [10, 20, 40, 50]
```

`pop(2)` menghapus elemen pada indeks 2 sekaligus mengembalikan nilai yang dihapus. Elemen setelah indeks tersebut bergeser ke kiri. Jika `pop()` dipanggil tanpa argumen, Python menghapus elemen terakhir, yang umumnya lebih murah karena tidak membutuhkan pergeseran elemen berikutnya.

### 2.6.2 `remove()` Menghapus Berdasarkan Nilai

**Contoh 2.8 Menghapus berdasarkan nilai**

```
data = [10, 20, 30, 20, 40]
data.remove(20)
print(data)
```

**Keluaran yang diharapkan:**

```
[10, 30, 20, 40]
```

`remove(20)` mencari kemunculan pertama nilai 20 kemudian menghapusnya. Artinya, operasi ini terdiri dari dua gagasan: mencari nilai lalu menggeser elemen setelah posisi tersebut. Jika nilai 20 muncul lebih dari satu kali, hanya kemunculan pertama yang dihapus.

### 2.6.3 `del` Menghapus Berdasarkan Posisi atau Slice

**Contoh 2.9 Menggunakan del**

```
data = [10, 20, 30, 40, 50]
del data[1]
print(data)
del data[1:3]
print(data)
```

**Keluaran yang diharapkan:**

```
[10, 30, 40, 50]
[10, 50]
```

`del data[1]` menghapus satu indeks. `del data[1:3]` menghapus rentang indeks 1 sampai sebelum 3. Sintaks *slice* mengikuti pola awal inklusif dan akhir eksklusif. Mahasiswa perlu berhati-hati karena penghapusan mengubah indeks elemen sesudahnya.

> Hindari menghapus elemen dari list yang sama sambil melakukan `for value in data` tanpa memahami akibatnya. Perubahan ukuran list ketika sedang ditelusuri dapat membuat elemen terlewat.

## 2.7. Traversal Array

Traversal berarti mengunjungi elemen secara sistematis. Pada array, traversal biasanya dilakukan dari indeks pertama menuju indeks terakhir. Jika seluruh n elemen harus dikunjungi sekali, kompleksitasnya O(n).

### 2.7.1 Traversal Berdasarkan Nilai

**Contoh 2.10 Traversal langsung terhadap nilai**

```
nilai = [78, 85, 90, 75, 88]
for x in nilai:
    print(x)
```

**Keluaran yang diharapkan:**

```
78
85
90
75
88
```

Variabel `x` menerima setiap elemen satu per satu. Bentuk ini sederhana jika program hanya membutuhkan nilai, bukan posisi indeks.

### 2.7.2 Traversal Berdasarkan Indeks

**Contoh 2.11 Traversal menggunakan indeks**

```
for i in range(len(nilai)):
    print(i, nilai[i])
```

**Keluaran yang diharapkan:**

```
0 78
1 85
2 90
3 75
4 88
```

`range(len(nilai))` menghasilkan urutan 0 sampai 4. Variabel `i` dipakai sebagai indeks. Bentuk ini berguna ketika algoritma memerlukan kontrol langsung terhadap posisi elemen.

### 2.7.3 Traversal dengan `enumerate()`

**Contoh 2.12 Traversal menggunakan enumerate()**

```
for i, x in enumerate(nilai):
    print('Indeks', i, 'nilai', x)
```

**Keluaran yang diharapkan:**

```
Indeks 0 nilai 78
Indeks 1 nilai 85
Indeks 2 nilai 90
Indeks 3 nilai 75
Indeks 4 nilai 88
```

`enumerate()` memberikan pasangan indeks dan nilai secara langsung. Dalam Python, bentuk ini sering lebih mudah dibaca daripada menggabungkan `range()` dan akses indeks. Linear Search pada modul ini akan memanfaatkan `enumerate()` agar hubungan antara posisi dan nilai terlihat jelas.

## 2.8. Linear Search

### 2.8.1 Ide Dasar

Linear Search adalah algoritma pencarian yang memeriksa elemen satu per satu dari awal hingga target ditemukan atau seluruh data selesai diperiksa. Algoritma ini tidak membutuhkan data terurut. Karena itu, Linear Search sangat fleksibel dan mudah diterapkan pada data kecil atau data yang belum diurutkan.




Gambar 2.3 Proses Linear Search untuk target 48

*Sumber: Diolah oleh penulis.*

### 2.8.2 Pseudocode

**Pseudocode 2.1 Linear Search**

```
LINEAR-SEARCH(data, target)
    untuk setiap indeks i dari 0 sampai n-1
        jika data[i] = target
            kembalikan i
    kembalikan -1
```

Nilai -1 digunakan sebagai penanda bahwa target tidak ditemukan. Pilihan ini umum digunakan ketika indeks valid dimulai dari 0, sehingga -1 dapat dibedakan dari posisi data yang sah.

### 2.8.3 Best Case dan Worst Case

| Kasus | Kondisi | Jumlah Pemeriksaan | Kompleksitas |
|---|---|---|---|
| Best case | Target berada pada elemen pertama | 1 | O(1) |
| Average case | Target rata-rata berada di sekitar tengah data | sekitar n/2 | O(n) |
| Worst case | Target terakhir atau tidak ada | n | O(n) |

## 2.9. Implementasi Linear Search dengan Python

**Contoh 2.13 Fungsi Linear Search**

```
def linear_search(data, target):
    for index, value in enumerate(data):
        if value == target:
            return index
    return -1
```

### 2.9.1 Penjelasan Kode Baris demi Baris

| Bagian Kode | Penjelasan |
|---|---|
| `def linear_search(data, target):` | Membuat fungsi dengan dua parameter: kumpulan data dan nilai yang dicari. |
| `for index, value in enumerate(data):` | Mengunjungi seluruh elemen sambil memperoleh indeks dan nilainya. |
| `if value == target:` | Membandingkan elemen saat ini dengan target. |
| `return index` | Menghentikan fungsi dan mengembalikan indeks ketika target ditemukan. |
| `return -1` | Dijalankan jika perulangan selesai tanpa menemukan target. |

### 2.9.2 Menjalankan Fungsi

**Contoh 2.14 Memanggil fungsi Linear Search**

```
data = [12, 35, 21, 48, 57, 63]

hasil1 = linear_search(data, 48)
hasil2 = linear_search(data, 99)

print('48 berada pada indeks:', hasil1)
print('99 berada pada indeks:', hasil2)
```

**Keluaran yang diharapkan:**

```
48 berada pada indeks: 3
99 berada pada indeks: -1
```

Pada pencarian pertama, program memeriksa 12, 35, 21, lalu 48. Begitu 48 ditemukan, `return index` menghentikan fungsi sehingga elemen sesudahnya tidak perlu diperiksa. Pada pencarian kedua, seluruh elemen diperiksa dan fungsi akhirnya mengembalikan -1.

### 2.9.3 Mencari Semua Kemunculan

Fungsi sebelumnya berhenti pada kemunculan pertama. Jika data dapat berisi duplikasi dan seluruh posisi harus ditemukan, algoritmanya harus tetap menelusuri sampai akhir.

**Contoh 2.15 Mencari seluruh kemunculan target**

```
def linear_search_all(data, target):
    posisi = []
    for index, value in enumerate(data):
        if value == target:
            posisi.append(index)
    return posisi

data = [20, 10, 20, 30, 20]
print(linear_search_all(data, 20))
```

**Keluaran yang diharapkan:**

```
[0, 2, 4]
```

Variabel `posisi` berupa list kosong yang digunakan untuk menampung seluruh indeks yang cocok. Karena algoritma harus memeriksa semua elemen meskipun target sudah ditemukan, kompleksitasnya tetap O(n).

## 2.10. Praktik Terbimbing: Melacak Linear Search

Sebelum masuk ke Binary Search, mahasiswa perlu terbiasa melakukan *dry run*, yaitu menjalankan algoritma secara manual di atas kertas atau tabel. Dry run membantu menemukan kesalahan logika tanpa bergantung pada komputer.

Gunakan data `[7, 11, 15, 18, 25, 30]` dan target `25`. Catat nilai variabel pada setiap iterasi.

| Iterasi | index | value | value == target? | Aksi |
|---|---|---|---|---|
| 1 | 0 | 7 | False | lanjut |
| 2 | 1 | 11 | False | lanjut |
| 3 | 2 | 15 | False | lanjut |
| 4 | 3 | 18 | False | lanjut |
| 5 | 4 | 25 | True | return 4 |

### 2.10.1 Menambahkan Jejak Eksekusi pada Program

**Contoh 2.16 Linear Search dengan jejak eksekusi**

```
def linear_search_trace(data, target):
    for index, value in enumerate(data):
        print('Periksa indeks', index, 'nilai', value)
        if value == target:
            print('Target ditemukan')
            return index
    print('Target tidak ditemukan')
    return -1

linear_search_trace([7, 11, 15, 18, 25, 30], 25)
```

**Keluaran yang diharapkan:**

```
Periksa indeks 0 nilai 7
Periksa indeks 1 nilai 11
Periksa indeks 2 nilai 15
Periksa indeks 3 nilai 18
Periksa indeks 4 nilai 25
Target ditemukan
```

Versi *trace* tidak diperlukan pada program produksi, tetapi sangat berguna untuk belajar. `print()` ditempatkan di dalam perulangan sehingga mahasiswa dapat melihat elemen mana yang sedang diperiksa dan kapan algoritma berhenti.

> Ketika memahami algoritma baru, biasakan membuat contoh input kecil, melakukan dry run, lalu membandingkan hasil manual dengan output program.

## 2.11. Binary Search

### 2.11.1 Syarat Utama: Data Harus Terurut

Binary Search mempercepat pencarian dengan menghilangkan sekitar setengah ruang pencarian pada setiap langkah. Strategi ini hanya valid jika data sudah terurut sesuai aturan yang konsisten, misalnya menaik dari nilai terkecil ke terbesar. Tanpa urutan, keputusan untuk membuang bagian kiri atau kanan tidak memiliki dasar.

Misalkan target adalah 57 dan data terurut `[12, 18, 25, 31, 44, 57, 63, 70, 82]`. Algoritma memeriksa elemen tengah. Jika nilai tengah lebih kecil dari target, seluruh bagian kiri termasuk nilai tengah dapat diabaikan. Jika lebih besar, bagian kanan dapat diabaikan.




Gambar 2.4 Penyempitan ruang pencarian pada Binary Search

*Sumber: Diolah oleh penulis.*

### 2.11.2 Variabel Penting

| Variabel | Makna |
|---|---|
| `left` | Indeks batas kiri ruang pencarian. |
| `right` | Indeks batas kanan ruang pencarian. |
| `mid` | Indeks tengah antara left dan right. |
| `data[mid]` | Nilai yang dibandingkan dengan target. |

### 2.11.3 Menghitung Indeks Tengah

**Rumus indeks tengah pada implementasi Python**

```
mid = (left + right) // 2
```

Operator `//` melakukan pembagian bulat ke bawah. Jika left = 0 dan right = 8, maka mid = 4. Setelah ruang pencarian berubah, nilai mid dihitung kembali berdasarkan batas terbaru.

## 2.12. Implementasi Binary Search dengan Python

**Contoh 2.17 Fungsi Binary Search iteratif**

```
def binary_search(data, target):
    left = 0
    right = len(data) - 1

    while left <= right:
        mid = (left + right) // 2

        if data[mid] == target:
            return mid
        elif data[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

### 2.12.1 Penjelasan Kode

| Bagian Kode | Fungsi |
|---|---|
| `left = 0` | Batas kiri dimulai pada indeks pertama. |
| `right = len(data) - 1` | Batas kanan dimulai pada indeks terakhir. |
| `while left <= right` | Perulangan berjalan selama ruang pencarian masih valid. |
| `mid = (left + right) // 2` | Menentukan posisi tengah. |
| `data[mid] == target` | Target ditemukan. |
| `data[mid] < target` | Target pasti berada di kanan, sehingga left digeser. |
| `data[mid] > target` | Target pasti berada di kiri, sehingga right digeser. |
| `return -1` | Ruang pencarian habis tanpa menemukan target. |

### 2.12.2 Menjalankan Fungsi

**Contoh 2.18 Pengujian Binary Search**

```
data = [12, 18, 25, 31, 44, 57, 63, 70, 82]

print(binary_search(data, 57))
print(binary_search(data, 13))
```

**Keluaran yang diharapkan:**

```
5
-1
```

Target 57 ditemukan pada indeks 5. Target 13 tidak ada sehingga fungsi mengembalikan -1. Perhatikan bahwa fungsi tidak mengurutkan data. Tanggung jawab memastikan data terurut berada pada pemanggil fungsi.

## 2.13. Dry Run Binary Search

Dry run Binary Search penting karena kesalahan kecil pada pembaruan `left` atau `right` dapat menghasilkan pencarian yang salah atau perulangan tidak berhenti. Gunakan data `[12, 18, 25, 31, 44, 57, 63, 70, 82]` dan target 57.

| Langkah | left | right | mid | data[mid] | Keputusan |
|---|---|---|---|---|---|
| 1 | 0 | 8 | 4 | 44 | 44 < 57 → left = 5 |
| 2 | 5 | 8 | 6 | 63 | 63 > 57 → right = 5 |
| 3 | 5 | 5 | 5 | 57 | ditemukan → return 5 |

### 2.13.1 Versi dengan Jejak Eksekusi

**Contoh 2.19 Binary Search dengan trace**

```
def binary_search_trace(data, target):
    left = 0
    right = len(data) - 1

    while left <= right:
        mid = (left + right) // 2
        print('left=', left, 'right=', right, 'mid=', mid, 'nilai=', data[mid])

        if data[mid] == target:
            return mid
        elif data[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

binary_search_trace([12,18,25,31,44,57,63,70,82], 57)
```

**Keluaran yang diharapkan:**

```
left= 0 right= 8 mid= 4 nilai= 44
left= 5 right= 8 mid= 6 nilai= 63
left= 5 right= 5 mid= 5 nilai= 57
```

Jejak tersebut menunjukkan bahwa dari sembilan elemen, target ditemukan hanya melalui tiga pemeriksaan. Pada data yang jauh lebih besar, pengurangan ruang pencarian

menjadi setengah pada setiap iterasi menghasilkan perbedaan yang sangat signifikan dibanding pemeriksaan satu per satu.

## 2.14. Mengapa Binary Search O(log n)?

Setiap langkah Binary Search membagi ukuran ruang pencarian kira-kira menjadi dua. Jika awalnya terdapat n elemen, setelah satu langkah tersisa sekitar n/2, kemudian n/4, n/8, dan seterusnya. Proses berhenti ketika hanya tersisa satu kemungkinan. Banyaknya pembagian dua inilah yang berkaitan dengan logaritma basis dua.

| n | Maksimum langkah kira-kira | Linear Search worst case |
|---|---|---|
| 8 | 3 | 8 |
| 16 | 4 | 16 |
| 32 | 5 | 32 |
| 1.024 | 10 | 1.024 |
| 1.048.576 | 20 | 1.048.576 |

Tabel menunjukkan bahwa ketika data berlipat ganda, jumlah langkah maksimum Binary Search hanya bertambah sekitar satu. Sebaliknya, pada Linear Search, jumlah pemeriksaan kasus terburuk juga ikut berlipat ganda.

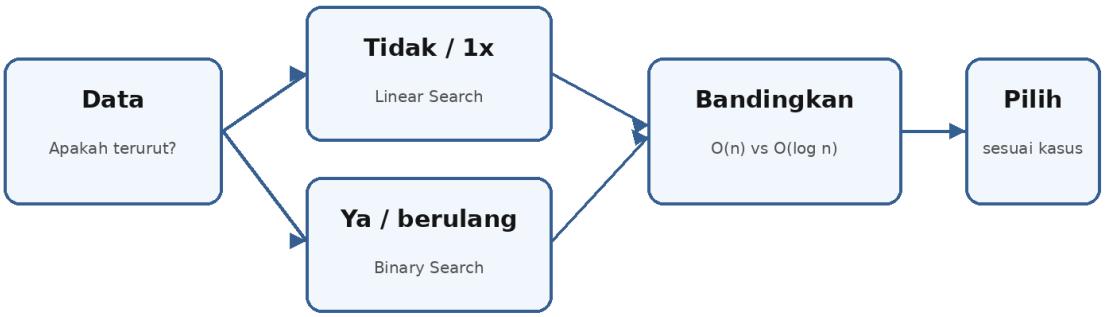
### 2.14.1 Tetapi Mengurutkan Data Juga Memiliki Biaya

Keunggulan O(log n) tidak berarti setiap data harus selalu diurutkan lalu dicari dengan Binary Search. Jika data belum terurut dan hanya akan dicari satu kali, proses mengurutkan data dapat menambah biaya yang tidak diperlukan. Sebaliknya, jika data relatif stabil dan akan dicari berkali-kali, biaya pengurutan awal dapat menjadi investasi yang masuk akal.

> Pemilihan algoritma harus mempertimbangkan keseluruhan proses, bukan hanya kompleksitas fungsi pencariannya. Pertanyaan penting: apakah data sudah terurut, seberapa sering pencarian dilakukan, dan seberapa sering data berubah?

## 2.15. Membandingkan Linear Search dan Binary Search

| Aspek | Linear Search | Binary Search |
|---|---|---|
| Syarat data | Tidak harus terurut | Harus terurut |
| Strategi | Periksa satu per satu | Buang setengah ruang pencarian |
| Best case | O(1) | O(1) |
| Worst case | O(n) | O(log n) |
| Implementasi | Sangat sederhana | Memerlukan batas left/right/mid |
| Cocok untuk | Data kecil, belum terurut, sedikit pencarian | Data terurut dan pencarian berulang |




Gambar 2.5 Pertimbangan awal memilih algoritma pencarian

*Sumber: Diolah oleh penulis.*

### 2.15.1 Contoh Keputusan

Kasus A: sebuah list berisi 20 nama peserta dan hanya akan dicari sekali. Linear Search sudah memadai karena implementasinya sederhana dan biaya pengurutan tambahan tidak diperlukan.

Kasus B: daftar 100.000 NIM sudah tersimpan terurut dan sistem melakukan ribuan pencarian. Binary Search menjadi pilihan yang lebih menarik karena setiap pencarian hanya membutuhkan sekitar log2(n) langkah pada kasus terburuk.

Kasus C: data terus berubah dan pencarian berdasarkan kunci dilakukan sangat sering. Pada kondisi seperti ini, struktur lain seperti *hash table* dapat lebih sesuai. Topik tersebut akan dibahas pada pertemuan berikutnya di semester ini.

## 2.16. Eksperimen Waktu Eksekusi di Google Colab

Analisis Big-O menjelaskan pola pertumbuhan secara teoretis. Untuk melengkapi pemahaman, mahasiswa dapat melakukan eksperimen sederhana menggunakan Python. Tujuannya bukan mendapatkan angka waktu yang dianggap absolut, tetapi mengamati kecenderungan perbedaan antara Linear Search dan Binary Search.

### 2.16.1 Menyiapkan Fungsi

Pastikan fungsi `linear_search()` dan `binary_search()` dari bagian sebelumnya sudah dijalankan di notebook. Setelah itu, jalankan cell berikut.

**Contoh 2.20 Eksperimen sederhana menggunakan timeit**

```
import timeit

data = list(range(100_000))
target = -1

waktu_linear = timeit.timeit(
    lambda: linear_search(data, target),
    number=100
)

waktu_binary = timeit.timeit(
    lambda: binary_search(data, target),
    number=100
)

print('Linear Search:', waktu_linear)
print('Binary Search:', waktu_binary)
```

### 2.16.2 Penjelasan Kode Eksperimen

| Kode | Makna |
|---|---|
| `list(range(100_000))` | Membuat data terurut 0 sampai 99.999. |
| `target = -1` | Target tidak ada sehingga Linear Search mengalami kasus terburuk. |
| `number=100` | Setiap fungsi dijalankan 100 kali agar waktu lebih mudah diukur. |
| `timeit.timeit(...)` | Mengukur total waktu eksekusi dari operasi yang diulang. |
| `lambda:` | Membungkus pemanggilan fungsi agar dapat dieksekusi oleh timeit. |

Angka yang muncul dapat berbeda pada setiap perangkat dan setiap sesi Colab. Yang diamati adalah pola: pada data besar dan target yang tidak ditemukan, Linear Search harus menelusuri seluruh data, sedangkan Binary Search hanya melakukan sedikit langkah.

Jangan menyimpulkan kompleksitas algoritma hanya dari satu kali pengukuran. Pengukuran empiris dipengaruhi perangkat, beban sistem, interpreter, cache, dan banyak faktor lain. Big-O tetap digunakan untuk menjelaskan pertumbuhan secara umum.

## 2.17. Eksperimen Ukuran Input

Agar hubungan ukuran input dan waktu lebih terlihat, ulangi pengukuran pada beberapa ukuran data. Kode berikut mencetak waktu untuk n = 1.000, 10.000, dan 100.000.

**Contoh 2.21 Mengamati pengaruh ukuran input**

```
for n in [1_000, 10_000, 100_000]:
    data = list(range(n))
    target = -1

    t_linear = timeit.timeit(
        lambda: linear_search(data, target),
        number=100
    )
    t_binary = timeit.timeit(
        lambda: binary_search(data, target),
        number=100
    )

    print('n =', n)
    print('  linear:', t_linear)
    print('  binary:', t_binary)
```

### 2.17.1 Cara Membaca Hasil

- Perhatikan apakah waktu Linear Search meningkat cukup nyata ketika n bertambah.
- Perhatikan bahwa waktu Binary Search biasanya bertambah jauh lebih lambat.
- Jangan membandingkan angka Anda dengan angka teman secara absolut karena lingkungan eksekusi dapat berbeda.
- Catat hasil ke dalam tabel dan tuliskan satu paragraf interpretasi.

### 2.17.2 Tabel Pengamatan

| n | Waktu Linear Search | Waktu Binary Search | Interpretasi Singkat |
|---|---|---|---|
| 1.000 | | | |
| 10.000 | | | |
| 100.000 | | | |

Tabel dikosongkan agar mahasiswa mengisi hasil dari notebook masing-masing. Fokus penilaian bukan memperoleh angka tertentu, tetapi mampu menjelaskan hubungan antara pertumbuhan input dan pola algoritma.

## 2.18. Studi Kasus: Pencarian Data Mahasiswa

Sebuah program sederhana menyimpan NIM mahasiswa. Kita ingin mengetahui apakah NIM tertentu terdaftar. Pertama, gunakan data yang belum terurut.

**Contoh 2.22 Linear Search pada NIM**

```
nim = ['41250017', '41250003', '41250029', '41250011', '41250008']
target = '41250011'

posisi = linear_search(nim, target)
if posisi != -1:
    print('NIM ditemukan pada indeks', posisi)
else:
    print('NIM tidak ditemukan')
```

**Keluaran yang diharapkan:**

```
NIM ditemukan pada indeks 3
```

Karena data belum terurut, Linear Search dapat digunakan langsung. Jika sistem kemudian ingin melakukan banyak pencarian, data dapat dipersiapkan dalam kondisi terurut.

**Contoh 2.23 Menyiapkan data untuk Binary Search**

```
nim_urut = sorted(nim)
print(nim_urut)

posisi = binary_search(nim_urut, target)
print('Posisi pada data terurut:', posisi)
```

Fungsi `sorted()` menghasilkan list baru yang sudah terurut. Indeks hasil Binary Search mengacu pada list `nim_urut`, bukan pada urutan awal. Hal ini penting jika indeks memiliki makna lain dalam sistem.

### 2.18.1 Pertanyaan Analisis

- Jika data hanya dicari satu kali, apakah mengurutkan data terlebih dahulu selalu menguntungkan? Jelaskan.
- Jika data ditambah setiap menit, apa konsekuensi mempertahankan urutan data?
- Jika NIM digunakan sebagai kunci pencarian utama dan pencarian sangat sering, struktur data apa yang mungkin lebih tepat daripada array?

## 2.19. Kesalahan Umum Saat Menggunakan Array dan Pencarian

### 2.19.1 IndexError

**Contoh kesalahan indeks di luar batas**

```
data = [10, 20, 30]
print(data[3])
```

List tersebut hanya memiliki indeks 0, 1, dan 2. Mengakses indeks 3 menyebabkan `IndexError: list index out of range`. Periksa `len(data)` dan pastikan indeks berada pada rentang yang valid.

### 2.19.2 Binary Search pada Data yang Belum Terurut

**Contoh penggunaan Binary Search yang tidak valid**

```
data = [40, 10, 30, 20, 50]
print(binary_search(data, 20))
```

Hasil fungsi tidak dapat dipercaya karena asumsi utama Binary Search dilanggar. Algoritma membuang bagian data berdasarkan perbandingan nilai tengah, sedangkan posisi nilai pada data belum terurut tidak mengikuti hubungan tersebut.

### 2.19.3 Kesalahan Batas Kanan

Jika menggunakan indeks inklusif seperti implementasi modul ini, batas kanan awal harus `len(data) - 1`, bukan `len(data)`. Nilai `len(data)` berada satu posisi setelah indeks terakhir.

### 2.19.4 Lupa Memperbarui Batas

Dalam Binary Search, `left` atau `right` harus berubah pada setiap iterasi ketika target belum ditemukan. Jika keduanya tidak berubah, nilai `mid` dapat tetap sama dan perulangan berpotensi tidak pernah berhenti.

### 2.19.5 Mengubah List Saat Traversal

Menghapus elemen ketika list sedang dilalui dari kiri ke kanan dapat mengubah posisi elemen yang belum diproses. Jika kebutuhan memang menghapus banyak elemen, pertimbangkan membuat list baru atau gunakan strategi iterasi yang sesuai.

## 2.20. Praktik Mandiri 2.1 — Operasi Array

Kerjakan seluruh langkah berikut pada notebook `SDA_Pertemuan_02_Nama_NIM.ipynb`. Tambahkan *text cell* singkat sebelum setiap bagian untuk menjelaskan apa yang sedang diuji.

1. Buat list `nilai` yang berisi minimal 10 nilai integer.
2. Tampilkan elemen pertama, elemen ke-5, dan elemen terakhir.
3. Ubah nilai pada salah satu indeks lalu tampilkan list setelah perubahan.
4. Tambahkan dua nilai menggunakan `append()`.
5. Sisipkan satu nilai pada indeks 2 menggunakan `insert()` dan amati perubahan posisi elemen.
6. Hapus satu elemen dengan `pop()` dan satu elemen dengan `remove()`.
7. Lakukan traversal menggunakan `enumerate()` sehingga setiap baris menampilkan indeks dan nilai.
8. Tulis komentar singkat di notebook mengenai operasi mana yang dapat menyebabkan pergeseran elemen.

### Checklist Hasil

- Notebook dapat dijalankan dari atas ke bawah tanpa error.
- Setiap bagian memiliki judul atau keterangan singkat.
- Output ditampilkan setelah operasi yang mengubah data.
- Mahasiswa dapat menjelaskan mengapa indeks berubah setelah insert/delete.

## 2.21. Praktik Mandiri 2.2 — Linear Search

Gunakan fungsi Linear Search buatan sendiri, bukan operator `in` atau method lain yang menyembunyikan proses pencarian.

1. Buat list minimal 15 bilangan yang tidak harus terurut.
2. Cari target yang berada pada elemen pertama dan catat jumlah pemeriksaan menggunakan fungsi trace.
3. Cari target yang berada pada elemen terakhir dan catat jumlah pemeriksaan.
4. Cari target yang tidak ada di dalam data dan catat jumlah pemeriksaan.
5. Bandingkan tiga kasus tersebut dan tentukan mana yang mewakili best case serta worst case.
6. Ubah fungsi agar mengembalikan pasangan `(index, jumlah_pemeriksaan)`.
7. Jelaskan dalam 3–5 kalimat mengapa worst case Linear Search adalah O(n).

### Tantangan Opsional

Modifikasi Linear Search agar pencarian teks tidak membedakan huruf besar dan kecil. Contoh: target `andi` harus dianggap sama dengan data `Andi`. Petunjuk: gunakan method string `lower()` pada saat perbandingan.

## 2.22. Praktik Mandiri 2.3 — Binary Search

1. Buat list berisi minimal 20 angka dan urutkan dengan `sorted()`.
2. Jalankan `binary_search_trace()` untuk target yang berada di sekitar tengah data.
3. Ulangi untuk target yang berada dekat awal dan dekat akhir data.
4. Ulangi untuk satu target yang tidak ada.
5. Catat nilai left, right, dan mid untuk salah satu pengujian dalam sebuah tabel.
6. Jalankan fungsi yang sama pada data yang sengaja diacak. Amati bahwa hasil tidak dapat dijadikan dasar dan jelaskan alasannya.
7. Bandingkan jumlah pemeriksaan Binary Search dengan Linear Search pada target yang sama.

Tujuan latihan bukan hanya mendapatkan indeks yang benar. Pastikan Anda dapat menjelaskan mengapa batas pencarian berubah pada setiap iterasi.

## 2.23. Mini Studi Kasus Terintegrasi

Sebuah kelas memiliki data nilai berikut. Dosen ingin mencari nilai tertentu, memperbarui satu nilai, menambahkan nilai mahasiswa baru, dan menghitung berapa kali suatu nilai muncul.

**Data kasus**

```
nilai = [65, 72, 88, 75, 90, 72, 81, 77, 95, 68]
```

### Tugas

1. Tampilkan seluruh data beserta indeksnya menggunakan `enumerate()`.
2. Cari nilai 90 menggunakan Linear Search dan tampilkan indeksnya.
3. Cari semua kemunculan nilai 72.
4. Ubah nilai pada indeks 0 menjadi 70.
5. Tambahkan nilai 84 di akhir dan sisipkan nilai 76 pada indeks 3.
6. Urutkan salinan data menggunakan `sorted()` tanpa mengubah data asli.
7. Cari nilai 88 pada data terurut menggunakan Binary Search.
8. Tuliskan kompleksitas utama untuk akses indeks, traversal, insert pada tengah, Linear Search, dan Binary Search.
9. Jelaskan algoritma pencarian mana yang Anda pilih jika pencarian hanya dilakukan sekali pada data belum terurut.
10. Jelaskan pilihan Anda jika data sudah terurut dan pencarian dilakukan ribuan kali.

### Luaran

Luaran berupa satu notebook yang dapat dijalankan dari awal sampai akhir. Setiap bagian harus menampilkan output dan diikuti penjelasan singkat dalam *text cell*. Penilaian lebih menekankan ketepatan logika dan kemampuan menjelaskan proses daripada banyaknya baris kode.

## 2.24. Ringkasan Kompleksitas Pertemuan 2

| Topik | Kompleksitas | Makna Intuitif |
|---|---|---|
| Akses `data[i]` | O(1) | Ukuran data tidak menambah jumlah langkah akses utama. |
| Update `data[i] = x` | O(1) | Posisi sudah diketahui. |
| Traversal | O(n) | Seluruh elemen dikunjungi. |
| Append | O(1) amortized | Biasanya langsung ke akhir; sesekali resize. |
| Insert/delete tengah | O(n) | Elemen lain dapat bergeser. |
| Linear Search best | O(1) | Target berada di awal. |
| Linear Search worst | O(n) | Target terakhir atau tidak ditemukan. |
| Binary Search best | O(1) | Target langsung berada di tengah. |
| Binary Search worst | O(log n) | Ruang pencarian dibagi dua setiap langkah. |

Tabel ini bukan untuk dihafalkan tanpa pemahaman. Setiap nilai kompleksitas harus dapat dijelaskan melalui proses yang dilakukan algoritma terhadap data.

## 2.25. Rangkuman

Array merupakan struktur data linear yang menyimpan elemen secara berurutan dan memungkinkan akses berdasarkan indeks. Pada Python, `list` digunakan sebagai implementasi praktis yang bersifat *dynamic array*. Akses dan update berdasarkan indeks bersifat O(1), sedangkan operasi yang menyebabkan pergeseran banyak elemen dapat bersifat O(n).

Linear Search dapat digunakan pada data yang belum terurut dan memiliki kompleksitas O(n) pada kasus terburuk. Binary Search membutuhkan data terurut, tetapi dapat memperkecil ruang pencarian menjadi setengah pada setiap langkah sehingga kompleksitas kasus terburuknya O(log n). Pemilihan algoritma tidak cukup berdasarkan angka Big-O saja; kondisi data, frekuensi pencarian, dan biaya persiapan data juga perlu dipertimbangkan.

Praktik Python pada pertemuan ini menekankan kemampuan menjalankan operasi array/list, melakukan traversal, menulis fungsi pencarian sendiri, melakukan dry run, membaca output, dan membandingkan analisis teoretis dengan eksperimen sederhana.

## 2.26. Latihan dan Evaluasi

1. Jelaskan perbedaan array statis dan *dynamic array*. Mengapa Python `list` lebih dekat dengan dynamic array?
2. Jika sebuah list mempunyai 12 elemen, berapa indeks elemen pertama dan terakhir?
3. Mengapa akses `data[i]` dianggap O(1), sedangkan traversal dianggap O(n)?
4. Jelaskan mengapa `insert(0, x)` dapat lebih mahal daripada `append(x)` pada list berukuran besar.
5. Tuliskan algoritma Linear Search dengan pseudocode Anda sendiri.
6. Apa perbedaan best case dan worst case Linear Search? Berikan contoh posisi target.
7. Mengapa Binary Search tidak boleh digunakan langsung pada data yang belum terurut?
8. Lakukan dry run Binary Search untuk data `[5, 12, 19, 27, 33, 41, 58, 64]` dengan target 41. Catat left, right, dan mid.
9. Bandingkan kompleksitas worst case Linear Search dan Binary Search. Jelaskan secara intuitif, bukan hanya menuliskan notasinya.
10. Data belum terurut berisi 50 elemen dan hanya akan dicari sekali. Algoritma apa yang paling sederhana digunakan? Jelaskan pertimbangannya.

11. Data terurut berisi satu juta elemen dan akan dicari ribuan kali. Mengapa Binary Search menjadi lebih menarik?
12. Buat fungsi Python `linear_search_all(data, target)` yang mengembalikan seluruh indeks target.
13. Modifikasi Binary Search agar selain indeks juga mengembalikan jumlah iterasi.
14. Jalankan eksperimen `timeit` untuk tiga ukuran data. Jelaskan tren yang Anda lihat dan hubungkan dengan Big-O.

### Latihan Diskusi

Sebuah aplikasi perpustakaan menyimpan 200.000 kode buku. Kode baru ditambahkan setiap hari dan pengguna sering melakukan pencarian. Diskusikan apakah mempertahankan data sebagai array terurut dengan Binary Search sudah memadai. Identifikasi keuntungan, biaya ketika melakukan penambahan data, serta alternatif struktur data yang mungkin dipelajari pada pertemuan selanjutnya.

## DAFTAR PUSTAKA


[1]. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms, 4th ed. MIT Press, 2022.

[2]. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. Data Structures and Algorithms in Python. John Wiley & Sons, 2013.

[3]. Bhargava, A. Y. Grokking Algorithms, Second Edition: An Illustrated Guide for Programmers and Other Curious People. Manning, 2024.

[4]. Miller, B. N., & Ranum, D. L. Problem Solving with Algorithms and Data Structures Using Python. Franklin, Beedle & Associates, 2011.

[5]. Python Software Foundation. The Python Tutorial — Data Structures. https://docs.python.org/3/tutorial/datastructures.html.

[6]. Python Software Foundation. `timeit` — Measure execution time of small code snippets. https://docs.python.org/3/library/timeit.html.

[7]. Google. Google Colaboratory. https://colab.research.google.com/.